

16_write_cpu.cpp - RDMA CPU内存写入演示

概述

此演示展示了使用AWS EFA（弹性网络适配器）和libfabric对CPU内存进行RDMA（远程直接内存访问）写入操作。它实现了客户端-服务器架构，其中服务器生成随机数据并使用RDMA操作直接写入客户端的CPU内存。

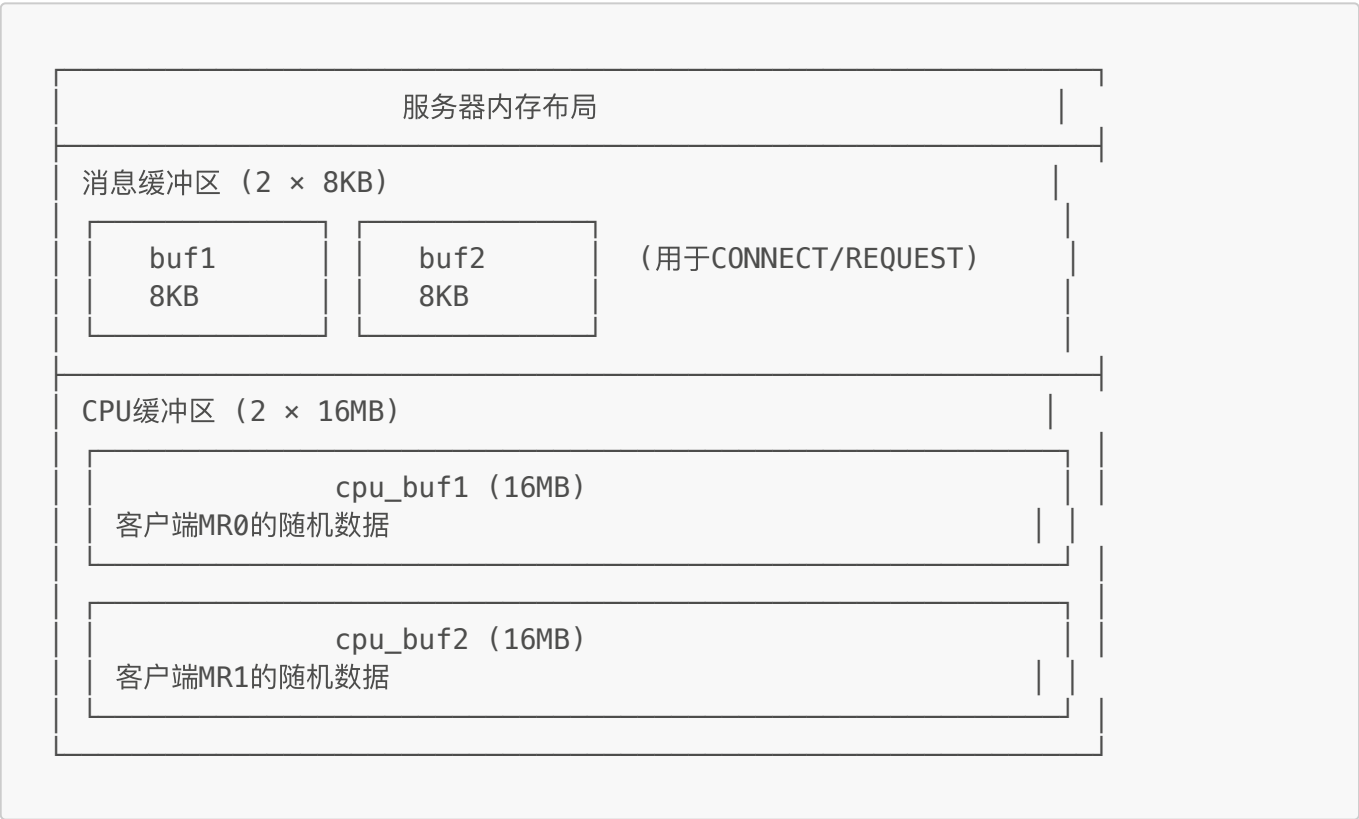
架构

演示采用客户端-服务器模型：

- **服务器**：生成随机数据并对客户端内存执行RDMA写入
- **客户端**：注册内存区域并通过RDMA写入接收数据

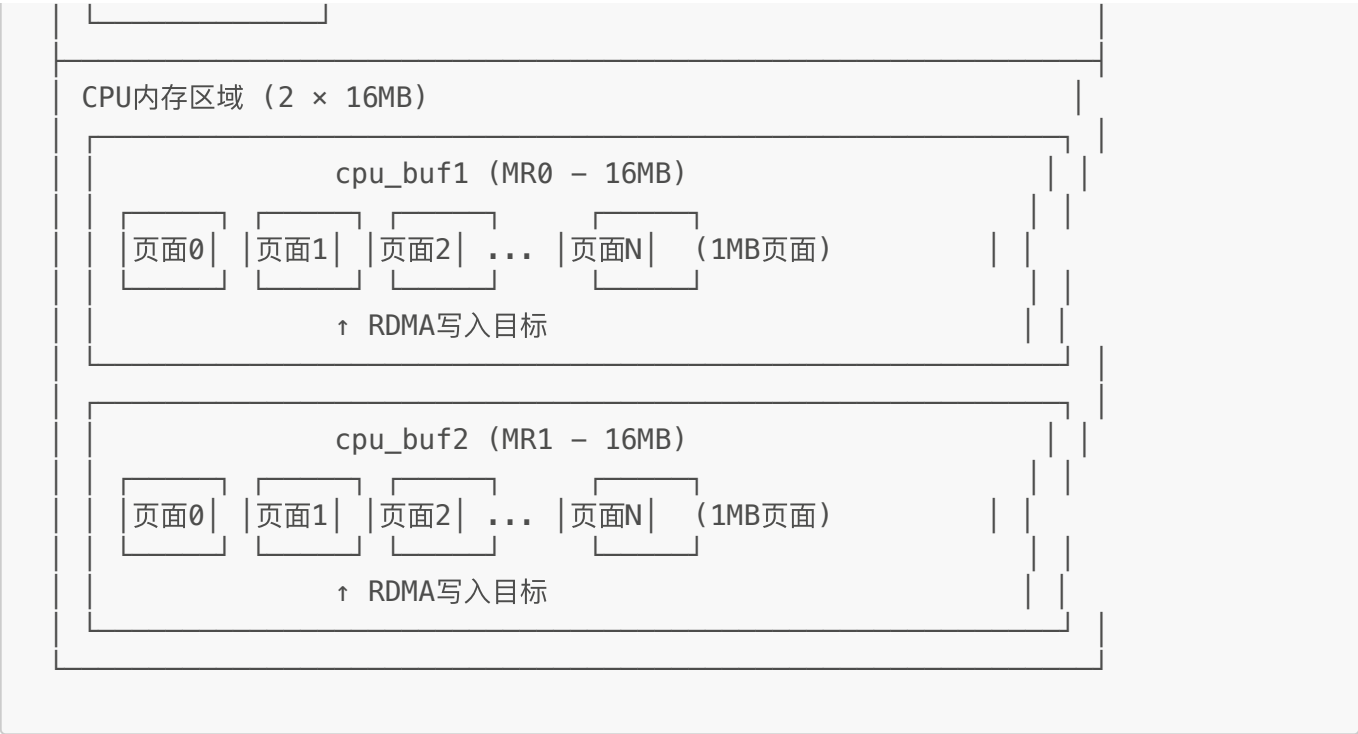
内存布局

服务器端内存



客户端内存





核心组件

核心数据结构

EfaAddress

- 表示32字节的EFA网络地址
- 提供字符串转换和解析工具
- 用于对等体识别和连接建立

Buffer

- 简化的内存缓冲区，无对齐要求
- 处理内存分配和清理
- 使用标准malloc/free进行内存管理

Network

- 包装libfabric操作的主要网络抽象
- 管理fabric、domain、完成队列、地址向量和端点
- 提供高级RDMA操作（发送、接收、写入）

RDMA操作

- **RdmaRecvOp**: 接收操作结构
- **RdmaSendOp**: 发送操作结构
- **RdmaWriteOp**: RDMA写入操作结构
- **RdmaRemoteWriteOp**: 远程写入完成跟踪

应用消息

AppConnectMessage

- 从客户端到服务器的初始握手消息
- 包含客户端地址和内存区域信息
- 包括用于RDMA访问的远程密钥（rkeys）

AppRandomFillMessage

- 随机数据生成的请求消息
- 指定种子、页面大小、页面数量
- 包含分散写入的页面索引

工作流程

服务器端

1. 初始化

- 打开libfabric网络接口
- 分配并注册两个独立的CPU内存缓冲区（每个16MB）
- 设置通信的消息缓冲区

2. 连接处理

- 接收来自客户端的CONNECT消息
- 提取客户端地址和内存区域信息
- 将客户端添加到地址向量

3. 数据生成和传输

- 接收带参数的RandomFill请求
- 使用指定种子生成随机数据
- 对客户端内存执行多个RDMA写入
- 使用立即数据进行完成通知

客户端

1. 初始化

- 打开libfabric网络接口
- 分配并注册两个CPU内存缓冲区
- 使用提供的地址连接到服务器

2. 内存注册

- 发送包含内存区域详细信息的CONNECT消息
- 为服务器访问提供远程密钥

3. 请求和验证

- 发送带参数的RandomFill请求
- 等待RDMA写入完成

- 验证接收到的数据与预期值

关键特性

RDMA写入操作

- 无需CPU参与的直接内存到内存传输
- 使用立即数据进行完成信号
- 处理多个内存区域和分散写入

错误处理

- 对EAGAIN错误实现指数退避的重试逻辑
- 对libfabric操作进行全面错误检查
- 资源的优雅清理

内存管理

- 无对齐要求的简化内存分配
- 用于RDMA访问的内存区域注册
- 通过RAII模式自动清理

调试功能

- 详细的页面索引跟踪和验证
- RDMA写入的边界检查
- 内存操作的全面日志记录
- 客户端配置显示

使用方法

服务器模式

```
./build/16_write_cpu
```

服务器显示其地址并等待客户端连接。

客户端模式

```
./build/16_write_cpu <server_address> [page_size num_pages]
```

- **server_address**: 来自服务器输出的64字符十六进制字符串
- **page_size**: 每个页面的大小（默认：1MB）
- **num_pages**: 要传输的页面数量（默认：8）

性能特征

- **内存大小**：每个内存区域16MB
- **默认传输**：8MB（8页面 × 1MB每页）
- **对齐**：无特殊对齐要求
- **并发性**：支持多个并发RDMA写入
- **调试输出**：用于故障排除的详细调试信息

技术细节

libfabric配置

- 提供商：EFA（弹性网络适配器）
- 端点类型：RDM（可靠数据报）
- 能力：MSG、RMA、LOCAL_COMM、REMOTE_COMM
- 内存注册：LOCAL、VIRT_ADDR、ALLOCATED、PROV_KEY

完成处理

- 使用数据完成队列格式
- 处理不同的完成类型（RECV、SEND、WRITE、REMOTE_WRITE）
- 实现基于回调的完成处理

数据验证

- 使用MT19937-64随机数生成器
- 基于确定性种子的生成用于验证
- 传输后比较预期与实际数据

错误场景

演示处理各种错误条件：

- 网络资源耗尽（EAGAIN）
- 无效地址或参数
- 内存注册失败
- 完成队列错误

依赖项

- libfabric（RDMA通信）
- EFA提供商（AWS弹性网络适配器）
- C++17标准库
- POSIX线程支持

数据流序列图（libfabric API级别）



```

|                                     |<-- fi_getinfo() ----->|
|                                     |-- info returned ----->|
|                                     |
|
|-- fi_fabric() ----->|
|-- fi_domain() ----->|
|-- fi_cq_open() ----->|
|-- fi_av_open() ----->|
|-- fi_endpoint() ----->|
|-- fi_ep_bind() ----->|
|-- fi_enable() ----->|
|
|                                     |<-- fi_fabric() ----->|
|                                     |<-- fi_domain() ----->|
|                                     |<-- fi_cq_open() ----->|
|                                     |<-- fi_av_open() ----->|
|                                     |<-- fi_endpoint() ----->|
|                                     |<-- fi_ep_bind() ----->|
|                                     |<-- fi_enable() ----->|
|
|-- fi_mr_regattr() ----->|                                     | // 注册
cpu_buf1
|-- fi_mr_regattr() ----->|                                     | // 注册
cpu_buf2
|-- fi_mr_regattr() ----->|                                     | // 注册
msg_buf
|                                     |<-- fi_mr_regattr() ----->| // 注册
cpu_buf1
|                                     |<-- fi_mr_regattr() ----->| // 注册
cpu_buf2
|                                     |<-- fi_mr_regattr() ----->| // 注册
msg_buf1
|                                     |<-- fi_mr_regattr() ----->| // 注册
msg_buf2
|
|                                     |<-- fi_recvmmsg() ----->| // 为
CONNECT投递接收
|                                     |<-- fi_recvmmsg() ----->| // 为
REQUEST投递接收
|
|-- fi_av_insert() ----->|                                     | // 添加服
务器地址
|
|== CONNECT消息交换 =====|=====|
|
|-- fi_sendmsg() ----->|                                     | // 发送
CONNECT
|   (AppConnectMessage)
|   - client_addr
|   - MR[0]: addr, size, rkey
|   - MR[1]: addr, size, rkey
|
|                                     |-- CONNECT已送达 ----->|
|                                     |
fi_cq_read() // 处理CONNECT
|                                     |<-- fi_av_insert() ----->| // 添加
客户端地址

```

```

|<-- 发送完成 -----|
|-- fi_cq_read() // 发送完成 |
|
|== 随机填充请求 =====|=====|
|
|-- AddRemoteWrite() ----->| // 设置
完成处理器
|
|-- fi_sendmsg() ----->| // 发送
RandomFill请求
| (AppRandomFillMessage)
| - remote_context: 0x123
| - seed: 0xb584...
| - page_size: 1048576
| - num_pages: 8
|
|----- REQUEST已送达 ----->|
|
fi_cq_read() // 处理REQUEST
|<-- 发送完成 -----|
|-- fi_cq_read() // 发送完成 |
|
|== 数据生成和RDMA写入 =====|=====|
|
RandomBytes() // 生成数据
|
|
fi_writemsg() // 写入MR0, 页面0
|
(imm_data=0)
|<-- RDMA WRITE -----|
|<-- 数据写入cpu_buf1 -----|
|
fi_writemsg() // 写入MR0, 页面1
|<-- RDMA WRITE -----|
|
(imm_data=0)
|<-- 数据写入cpu_buf1 -----|
|
...
fi_writemsg() // 写入MR1, 页面0
|<-- RDMA WRITE -----|
|
(imm_data=0)
|<-- 数据写入cpu_buf2 -----|
|
...
fi_writemsg() // 最终写入
|<-- RDMA WRITE -----|
|
(imm_data=0x123)
|<-- 数据写入cpu_buf2 -----|
|<-- REMOTE_WRITE完成 -----|
| (cq.data=0x123)
|-- fi_cq_read() // 最终完成
|
fi_cq_read() // 写入完成

```

```

|== 数据验证 =====|=====|
|
|-- RandomBytes() // 生成预期数据
|-- memcmp() // 验证数据
|-- printf("数据正确")
|

```

关键libfabric API调用说明：

初始化阶段：

- `fi_getinfo()`：查询可用的fabric提供商和能力
- `fi_fabric()`、`fi_domain()`：创建fabric和domain对象
- `fi_cq_open()`、`fi_av_open()`：创建完成队列和地址向量
- `fi_endpoint()`、`fi_ep_bind()`、`fi_enable()`：创建并激活端点

内存注册：

- `fi_mr_regattr()`：注册内存区域以供RDMA访问
- 返回用于远程访问的内存密钥（rkeys）

通信设置：

- `fi_recvmmsg()`：投递接收缓冲区（服务器端）
- `fi_av_insert()`：将对等地址添加到地址向量

消息交换：

- `fi_sendmsg()`：发送消息（CONNECT、RandomFill请求）
- `fi_cq_read()`：轮询完成队列以获取操作完成

RDMA操作：

- `fi_writemsg()`：对远程内存执行RDMA写入
- 使用来自CONNECT消息的远程地址、长度和rkey
- 最终写入包含用于完成通知的立即数据

完成处理：

- 服务器获得本地写入完成
- 客户端获得带立即数据的远程写入完成
- `FI_REMOTE_WRITE`标志表示远程写入完成

该序列展示了libfabric如何实现应用程序内存区域之间的零拷贝、内核旁路数据传输。

此演示有效地展示了在AWS环境中使用EFA进行CPU内存传输的高性能RDMA操作。

其他 引用文章 <https://www.slideshare.net/slideshow/ofi-libfabric-tutorial/55227620>